

Task 1

```
CSVReader::CSVReader() {}

std::vector<WeatherRow> CSVReader::readWeatherCSV(const std::string&
csvFilename) {
    std::vector<WeatherRow> data;
    std::ifstream csvFile(csvFilename);
    std::string line;

    if (!csvFile.is_open()) {
        throw std::runtime_error("Could not open file: " + csvFilename);
    }

    // Get Country Code
    std::vector<std::string> headers;
    if (std::getline(csvFile, line)) {
        headers = tokenise(line, ',');
        for (auto& header : headers) {
            if (header.size() > 12 && header.substr(header.size() - 12) ==
"_temperature") {
                header = header.substr(0, header.size() - 12);
            }
        }
    }

    while (std::getline(csvFile, line)) {
        auto tokens = tokenise(line, ',');
        if (tokens.size() != headers.size()) {
            std::cerr << "CSVReader::readWeatherCSV - Missing columns in line:
" << line << std::endl;
            continue;
        }

        WeatherRow row;
        row.timestamp = tokens[0];
        try {
            for (size_t i = 1; i < tokens.size(); ++i) {
                row.values[headers[i]] = std::stod(tokens[i]);
            }
            data.push_back(row);
        } catch (const std::exception& e) {
            std::cerr << "CSVReader::readWeatherCSV - Failed to convert in
line: " << line << std::endl;
        }
    }
}
```

```
    return data;
}
```

In CSVReader.cpp file, the readWeatherCSV function look for the csvFile, open it and get the column header for country code like GB, EE. Then it get the weather data and store them as double data type.

```
int main() {
    WeatherApp app;
    app.run();
    return 0;
}
```

In my main file, the run function from WeatherApp c++ file is called and readWeatherCSV is the first function that start running inside the run function.

```
void WeatherApp::run() {
    try {
        std::cout << "\nWelcome to TT Weather App!" << std::endl;
        auto data = CSVReader::readWeatherCSV("weather.csv");
    }
}
```

So everything above is running by passing “weather.csv” as parameter.

```
std::vector<std::string> CSVReader::tokenise(const std::string& csvLine, char
separator) {
    std::vector<std::string> tokens;
    size_t start = 0, end = 0;

    while ((end = csvLine.find(separator, start)) != std::string::npos) {
        tokens.push_back(csvLine.substr(start, end - start));
        start = end + 1;
    }

    tokens.push_back(csvLine.substr(start));
    return tokens;
}
```

The tokensie function is also called which is to chop the data by the separator (for separating data in csv would be comma) like how we have taught in coursera.

```
Candlestick::Candlestick(std::string date, double open, double high, double low, double close)
    : date(date), open(open), high(high), low(low), close(close) {}
```

Inside the Candlestick c++ file is the candlestick constructor which will store date, open, high, low, close value

```
std::vector<Candlestick> WeatherAnalyzer::computeCandlestickData(
    const std::vector<WeatherRow>& data, const std::string& country) {
    std::vector<Candlestick> candlesticks;
    std::map<int, std::vector<double>> yearlyData;

    // Group temperatures by year
    for (const auto& entry : data) {
        try {
            int year = std::stoi(entry.timestamp.substr(0, 4));
            if (entry.values.find(country) != entry.values.end()) {
                yearlyData[year].push_back(entry.values.at(country));
            }
        } catch (const std::exception& e) {
            std::cerr << "Error processing with timestamp: " <<
entry.timestamp << std::endl;
        }
    }

    double previousClose = 0.0;
    for (int year = 1980; year <= 2019; ++year) {
        double open = previousClose, high = 0.0, low = 0.0, close = 0.0;

        if (yearlyData.count(year)) {
            computeStatistics(yearlyData[year], high, low, close);
        }

        candlesticks.emplace_back(std::to_string(year), open, high, low,
close);
        previousClose = close;
    }

    return candlesticks;
}
```

I use the constructor in computeCandlestickData function to get the yearly open, high, low close, value.

```
void WeatherAnalyzer::computeStatistics(const std::vector<double>& values,
double& high, double& low, double& mean) {
    if (!values.empty()) {
```

```

        high = *std::max_element(values.begin(), values.end());
        low = *std::min_element(values.begin(), values.end());
        mean = std::accumulate(values.begin(), values.end(), 0.0) /
values.size();
    } else {
        high = low = mean = 0.0;
    }
}

```

For processing with the result data like finding highest for high, mean data for close, I create computeStatistics function.

After getting the result , the computerCandlestickData function return the constructor with correct value. Then the value is displayed in the terminal by this code.

```

void WeatherApp::displayCandlesticks(const std::vector<Candlestick>&
candlesticks, int rangeType) const {
    std::cout << std::setw(10) << "Date" << std::setw(10) << "Open"
        << std::setw(10) << "High" << std::setw(10) << "Low"
        << std::setw(10) << "Close" << std::endl;

    // Task 1
    for (const auto& candle : candlesticks) {
        std::cout << std::setw(10) << candle.date
            << std::setw(10) << std::fixed << std::setprecision(2) <<
candle.open
            << std::setw(10) <<
static_cast<int>(std::round(candle.high))
            << std::setw(10) << static_cast<int>(std::round(candle.low))
            << std::setw(10) << std::fixed << std::setprecision(2) <<
candle.close
            << std::endl;
    }
}

```

```

void WeatherApp::filterByYear(const std::vector<WeatherRow>& data) {
    auto candlesticks = WeatherAnalyzer::computeCandlestickData(data,
selectedCountry);

    CSVReader::adjustFirstOpenValue(candlesticks, 1);

    // Display historical candlesticks
    displayCandlesticks(candlesticks, 1);
}

```

CSVReader::adjustFirstOpenValue(candlesticks, 1) is to adjust the first open value since there is no data for that. When I first did Task 1, there is no second parameter as I only tried for yearly data and that parameter is now used for filtering.

```
void CSVReader::adjustFirstOpenValue(std::vector<Candlestick>& candlesticks,
int rangeType) {
    const std::string& firstDate = candlesticks[0].date; // Extract once for reuse

    switch (rangeType) {
        case 1: // Yearly
            if (firstDate.substr(0, 4) == "1980") {
                candlesticks[0].open = candlesticks[0].close + 0.4;
            }
            break;

        case 2: // Monthly
            if (firstDate.substr(0, 7) == "1980-01") {
                candlesticks[0].open = candlesticks[0].close + 0.4;
            }
            break;

        case 3: // Daily
            // Every first day
            if (firstDate.size() >= 10 && firstDate.substr(8, 2) == "01") {
                candlesticks[0].open = candlesticks[0].close + 0.4;
            }
            break;
    }
}
```

The function is from CSVReader c++ file and it is just to apply + 0.4 of current mean value for previous mean value

After everything together, I have successfully done my task 1 and this is the example result.

I forget to mention that I had round up the value to two decimal for better view and for task 2

```
Available Country Code for weather data: AT BE BG CH CZ DE DK EE ES FI FR GB GR HR HU IE IT LT LU LV NL NO PL PT RO SE SI SK
Enter the country code (or type 'exit' to quit, 'b' to go back): NL
```

```
Select date range type:
```

1. Yearly
2. Monthly
3. Daily
4. Back to Country Selection

```
Enter choice (from 1 to 4 or type 'exit' to quit): 1
```

Date	Open	High	Low	Close
1980	9.47	27	-9	9.07
1981	9.07	27	-11	9.15
1982	9.15	27	-11	9.68
1983	9.68	29	-8	9.76
1984	9.76	27	-6	9.12
1985	9.12	26	-14	8.21
1986	8.21	27	-11	8.63
1987	8.63	26	-14	8.42
1988	8.42	25	-3	9.96
1989	9.96	28	-3	10.36
1990	10.36	31	-3	10.53
1991	10.53	29	-12	9.35
1992	9.35	29	-6	10.18
1993	10.18	27	-9	9.37
1994	9.37	31	-8	10.29
1995	10.29	29	-10	10.10
1996	10.10	30	-12	8.17
1997	8.17	30	-14	9.89
1998	9.89	29	-7	10.05
1999	10.05	28	-4	10.63
2000	10.63	29	-4	10.58
2001	10.58	29	-7	10.03
2002	10.03	30	-9	10.40
2003	10.40	30	-10	10.12
2004	10.12	29	-4	10.13
2005	10.13	30	-8	10.36
2006	10.36	31	-7	10.72
2007	10.72	27	-5	10.92
2008	10.92	28	-6	10.26
2009	10.26	29	-10	10.17
2010	10.17	30	-11	8.65
2011	8.65	28	-5	10.49
2012	10.49	30	-14	9.93
2013	9.93	31	-8	9.37
2014	9.37	30	-5	11.32
2015	11.32	29	-3	10.44
2016	10.44	30	-5	10.40
2017	10.40	28	-6	10.66
2018	10.66	32	-8	11.03
2019	11.03	35	-5	11.11

Task 2

```
if (candlesticks.empty()) {
    std::cerr << "No candlestick data found for the selected range." <<
std::endl;
} else {
    // Task 2 with fixed range graph
    std::cout << " " << std::endl;
    std::cout << "Candlestick Plot:" << std::endl;
    CandlestickPlotter::plot(candlesticks, -12.0, 30.0, rangeType);
}
}
```

Task 2 is mainly the plot drawing function and this line of code combine with task 1 under displayCandlestick function to show together

```

void CandlestickPlotter::plot(const std::vector<Candlestick>& candlesticks,
double minRange, double maxRange, int rangeType) {
    if (candlesticks.empty()) {
        std::cerr << "No data available for plotting." << std::endl;
        return;
    }

    const int plotHeight = 30; // Number of rows for the vertical plot

    // Draw Graph based on these values
    double maxHigh = std::numeric_limits<double>::lowest();
    double minLow = std::numeric_limits<double>::max();
    double maxOpen = std::numeric_limits<double>::lowest();
    double minOpen = std::numeric_limits<double>::max();

    for (const auto& candle : candlesticks) {
        maxHigh = std::max(maxHigh, candle.high);
        minLow = std::min(minLow, candle.low);
        maxOpen = std::max(maxOpen, candle.open);
        minOpen = std::min(minOpen, candle.open);
    }

    // Add padding to high and low values for more accuracy
    double graphMax = maxHigh + 5;
    double graphMin = minLow - 5;

    // Generate axis labels
    std::vector<double> axisLabels;

    // Top values range
    for (double val = graphMax; val >= maxOpen; val -= 1.5) {
        axisLabels.push_back(val);
    }

    // Top values range
    for (double val = maxOpen; val > minOpen; val -= 0.2) {
        axisLabels.push_back(val);
    }

    // Bottom values range
    for (double val = minOpen - 0.1; val >= graphMin; val -= 1.5) {
        axisLabels.push_back(val);
    }

    // Generate the plot grid
    std::vector<std::string> plot(axisLabels.size(),
std::string(candlesticks.size() * 6, ' '));

```

```

for (size_t i = 0; i < candlesticks.size(); ++i) {
    const auto& candle = candlesticks[i];

    auto findPosition = [&](double value) {
        auto it = std::lower_bound(axisLabels.begin(), axisLabels.end(),
value, std::greater<>());
        return std::distance(axisLabels.begin(), it);
    };

    int openPos = std::clamp(static_cast<int>(findPosition(candle.open)),
        0, static_cast<int>(axisLabels.size() - 1));
    int closePos =
std::clamp(static_cast<int>(findPosition(candle.close)),
        0, static_cast<int>(axisLabels.size() - 1));
    int highPos = std::clamp(static_cast<int>(findPosition(candle.high)),
        0, static_cast<int>(axisLabels.size() - 1));
    int lowPos = std::clamp(static_cast<int>(findPosition(candle.low)),
        0, static_cast<int>(axisLabels.size() - 1));

    // Draw high and low (stalk)
    for (int j = highPos; j <= lowPos; ++j) {
        plot[j][i * 6 + 3] = '|';
    }

    // Draw open marks
    plot[openPos][i * 6 + 2] = '(';
    plot[openPos][i * 6 + 3] = 'o';
    plot[openPos][i * 6 + 4] = ')';

    // Draw close marks
    plot[closePos][i * 6 + 2] = '(';
    plot[closePos][i * 6 + 3] = 'c';
    plot[closePos][i * 6 + 4] = ')';
}

// Print the plot with vertical temperature axis
for (size_t j = 0; j < axisLabels.size(); ++j) {
    double value = axisLabels[j];
    std::cout << std::setw(6) << std::fixed << std::setprecision(1) <<
value << " |";

    for (size_t k = 0; k < plot[j].size(); ++k) {
        std::cout << plot[j][k];
    }
    std::cout << std::endl;
}

// Print the horizontal axis

```

```

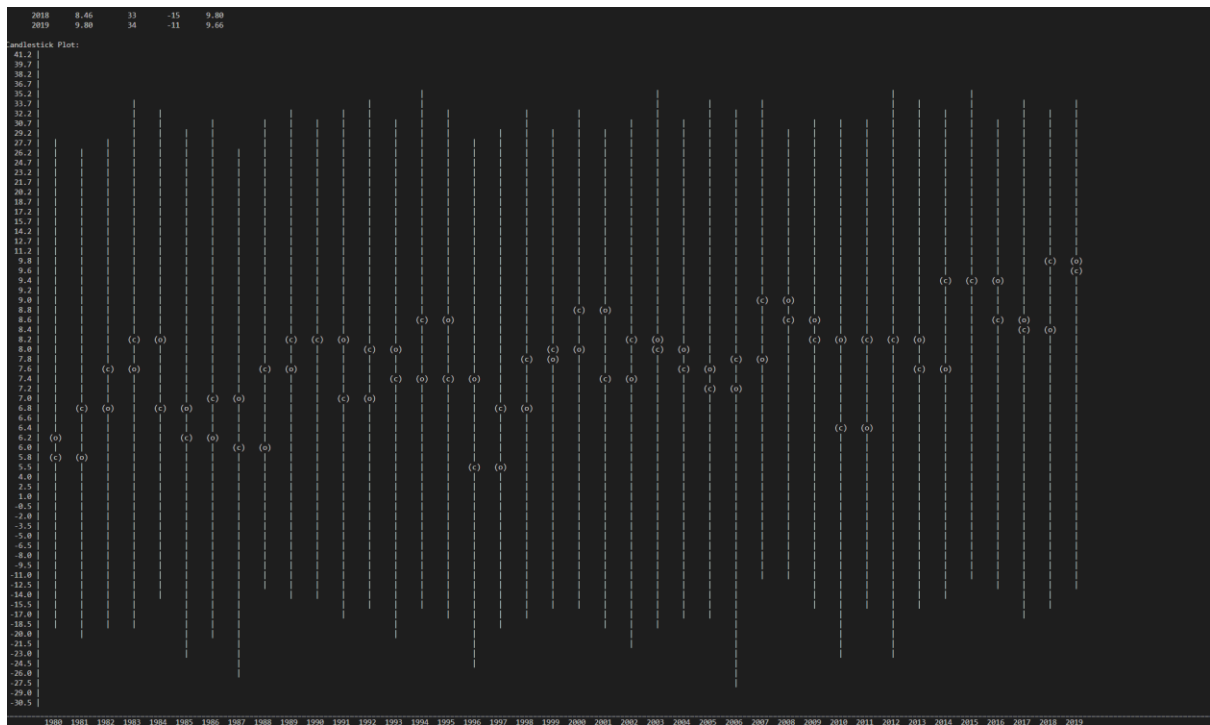
std::cout << std::string(candlesticks.size() * 7, '_') << std::endl;
std::cout << "          ";
for (size_t i = 0; i < candlesticks.size(); ++i) {
    // Yearly
    if (rangeType == 1) {
        // more entries than other 2
        std::cout << std::setw(6) << candlesticks[i].date.substr(0, 4);
    }
    // Monthly and daily
    else if (rangeType == 2 || rangeType == 3) {
        std::cout << std::setw(6) << (i + 1);
    }
}
std::cout << std::endl;
}

```

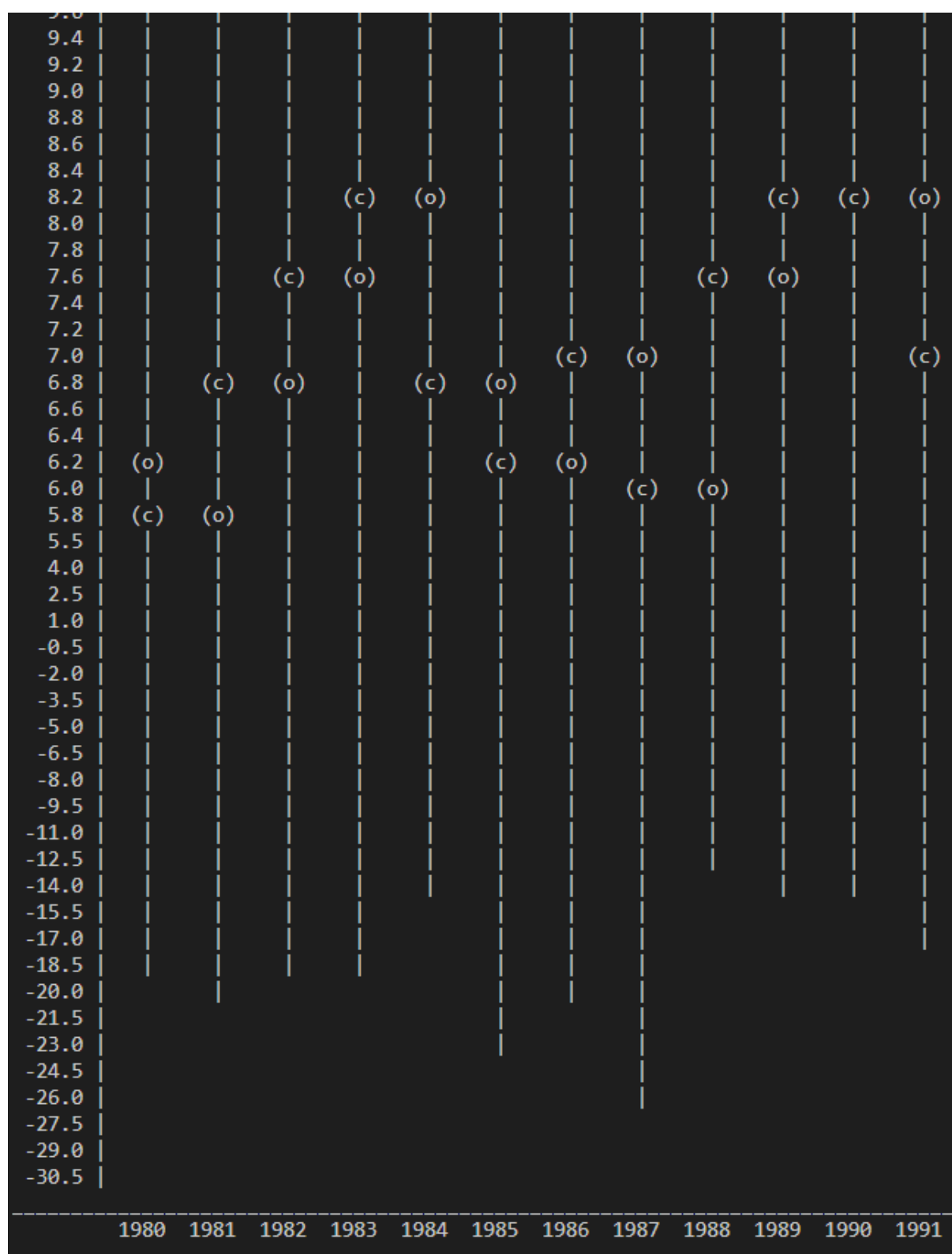
To determine how I am going to draw the graph, I look through the data and I found that the high and low values have huge differences than mean value. And I also thought the temperature would be in the range of 30 to -12 and I set a fixed range for the graph but turns out some countries exceed the range.

So I change the graph to be dynamic by detecting the maximum high, minimum low value for the graph range and highest and lowest mean value for middle part of the graph. The middle part is included as if we do the graph with normal padding, the differences in close and open values couldn't be seen. As you can see, I set 1.5 different for normal padding and 0.2 for finding the open, close values.

I also included the label, and vertical line shows the value range while horizontal line shows the date range



This is the overall plot with the candlestick. I used (o) for open value and (c) for close value in the graph.



Task 3

In task 3, we are asked to do at least 2 filtering and this is when I started implementing the menu since there is no way to do 2 filtering without the menu.

I had expanded my WeatherApp.cpp file and now it has selectCountry function, which is similar to filtering by country. The function gets the country input from the user and made decision based on that.

```
// Choose Country
// Task 3, filter by country
bool WeatherApp::selectCountry(const std::vector<WeatherRow>& data) {
    std::set<std::string> countrySet;
    for (const auto& header : data[0].values) {
        // Get the country code
        countrySet.insert(header.first);
    }

    while (true) { // Loop until valid input is received
        std::cout << "Available Country Code for weather data: ";
        for (const auto& country : countrySet) {
            std::cout << country << " ";
        }
        std::cout << std::endl;

        std::string country;
        std::cout << "Enter the country code (or type 'exit' to quit, 'b' to go back): ";
        std::cin >> country;

        if (country == "exit") {
            std::cout << "Program exit. Thank you for using!" << std::endl;
            exit(0);
        } else if (country == "b") {
            return false;
        }

        if (countrySet.find(country) != countrySet.end()) {
            selectedCountry = country;
            return true;
        }

        std::cerr << "Invalid country code. Please try again or type 'exit' to quit, 'b' to go back." << std::endl;
    }
}
```

```
}  
}
```

If we go back to the run function, I had added menu layout with 2 options for the user which are displaying the data and the prediction of data. Since we are still at task 3, I will skip the second option

```
void WeatherApp::run() {  
    try {  
        std::cout << "\nWelcome to TT Weather App!" << std::endl;  
        auto data = CSVReader::readWeatherCSV("weather.csv");  
  
        while (true) {  
            std::string mode;  
            std::cout << "Select mode:\n"  
                << "1. Display Data\n"  
                << "2. Predict Yearly Data\n"  
                << "Enter choice (1 or 2, or 'exit' to quit): ";  
            std::cin >> mode;  
  
            if (mode == "exit") {  
                std::cout << "Program exit, thank you for using!" <<  
std::endl;  
                exit(0);  
            } else if (mode == "1") {  
                handleDataDisplay(data);  
            } else if (mode == "2") {  
                // Task 4  
                handlePrediction(data);  
            } else {  
                std::cerr << "Invalid choice. Please enter 1, 2, or 'exit'."  
<< std::endl;  
            }  
        }  
    } catch (const std::exception& e) {  
        std::cerr << "Error: " << e.what() << std::endl;  
    }  
}
```

```
void WeatherApp::handleDataDisplay(const std::vector<WeatherRow>& data) {  
    while (true) {  
        if (!selectCountry(data)) {  
            return;  
        }  
  
        // Directly display the menu
```

```

        runMenu(data);
    }
}

```

```

// Runs the menu
void WeatherApp::runMenu(const std::vector<WeatherRow>& data) {
    while (true) {
        std::string input;
        // Task 3, filter by date range
        std::cout << std::endl << "Select date range type:" << std::endl
            << "1. Yearly" << std::endl
            << "2. Monthly" << std::endl
            << "3. Daily" << std::endl
            << "4. Back to Country Selection" << std::endl;

        while (true) {
            std::cout << "Enter choice (from 1 to 4 or type 'exit' to quit):
";
            std::cin >> input;

            if (input == "exit") {
                std::cout << "Program exit, thank you for using!" <<
std::endl;
                exit(0); // Terminate the program
            }

            int choice;
            try {
                choice = std::stoi(input);
                if (choice >= 1 && choice <= 4) {
                    break;
                } else {
                    std::cerr << "Invalid choice. " << std::endl;
                }
            } catch (const std::invalid_argument&) {
                std::cerr << "Invalid input. " << std::endl;
            }
        }

        int choice = std::stoi(input);
        switch (choice) {
            case 1:
                filterByYear(data);
                break;
            case 2:
                filterByMonth(data);
                break;

```

```

        case 3:
            filterByDay(data);
            break;
        case 4:
            std::cout << " " << std::endl;
            std::cout << "Back to country selection" << std::endl;
            return;
    }
}
}

```

This is how my program work with these 3 functions. First it will welcome the user and ask the option 1 or 2. If the user choose one, it will direct to display page with the help of hadleDataDisplay then runMenu function. In runMenu function, you will see the second filtering option, which is to filter by date range. I also added error handling while asking input from the user to prevent wrong entry like random string.

Since, I have already done the yearly data in task 2, I will go through monthly and daily data display

```

void WeatherApp::filterByMonth(const std::vector<WeatherRow>& data) {
    std::string input;
    int year;

    while (true) {
        std::cout << "Enter year (1980-2019) or type 'exit' to quit: ";
        std::cin >> input;

        if (input == "exit") {
            std::cout << "Program exit, thank you for using!" << std::endl;
            exit(0);
        }

        try {
            year = std::stoi(input);
            if (year >= 1980 && year <= 2019) {
                break;
            } else {
                std::cerr << "Invalid year. Please enter a year between 1980
and 2019." << std::endl;
            }
        } catch (const std::invalid_argument&) {
            std::cerr << "Invalid input. Please enter a valid year or type
'exit' to quit." << std::endl;
        }
    }
}

```

```

    }

    auto monthlyCandlesticks =
WeatherAnalyzer::computeMonthlyCandlestickData(data, selectedCountry, year);

    if (monthlyCandlesticks.empty()) {
        std::cerr << "No data found for the given year and country." <<
std::endl;
        return;
    }

    CSVReader::adjustFirstOpenValue(monthlyCandlesticks, 2);
    displayCandlesticks(monthlyCandlesticks, 2);
}

```

For filtering the month, the system need to request user about which year monthly data he wants to see. Then after successful input, it will use computeMonthlyCandlestickData to calculate the data.

```

// Computes monthly candlestick data
std::vector<Candlestick> WeatherAnalyzer::computeMonthlyCandlestickData(
    const std::vector<WeatherRow>& data,
    const std::string& country,
    int year) {
    std::vector<Candlestick> candlesticks;
    std::map<int, std::vector<double>> monthlyData;

    // Group data by month
    for (const auto& entry : data) {
        int entryYear = std::stoi(entry.timestamp.substr(0, 4));
        int entryMonth = std::stoi(entry.timestamp.substr(5, 2));
        if (entryYear == year && entry.values.find(country) !=
entry.values.end()) {
            monthlyData[entryMonth].push_back(entry.values.at(country));
        }
    }

    double previousClose = 0.0;

    for (int month = 1; month <= 12; ++month) {
        double open = 0.0;
        double close = 0.0;
        double high = std::numeric_limits<double>::lowest();
        double low = std::numeric_limits<double>::max();

        if (monthlyData.find(month) != monthlyData.end()) {
            const auto& temperatures = monthlyData[month];

```

```

        // Compute close (current month's mean)
        close = std::accumulate(temperatures.begin(), temperatures.end(),
0.0) / temperatures.size();

        // Compute high and low
        high = *std::max_element(temperatures.begin(),
temperatures.end());
        low = *std::min_element(temperatures.begin(), temperatures.end());
    }

    // Set the open value
    if (month == 1) {
        if (year == 1980) {
            // Since there is no data for 1979, 1980 Jan open value is set
to close value + 0.4
            open = close + 0.4;
        } else {
            // For other years, use December mean of the previous year
            std::string prevDecember = std::to_string(year - 1) + "-12";
            std::vector<double> prevYearTemps;
            for (const auto& entry : data) {
                if (entry.timestamp.substr(0, 7) == prevDecember &&
                    entry.values.find(country) != entry.values.end()) {
                    prevYearTemps.push_back(entry.values.at(country));
                }
            }
            if (!prevYearTemps.empty()) {
                open = std::accumulate(prevYearTemps.begin(),
prevYearTemps.end(), 0.0) / prevYearTemps.size();
            } else {
                open = close + 0.4;
            }
        }
    } else {
        // For other months, use the previous month's close
        open = previousClose;
    }

    // Add the candlestick for the month
    candlesticks.emplace_back(
        std::to_string(year) + "-" + (month < 10 ? "0" : "") +
std::to_string(month),
        open, high, low, close);

    // Update previousClose for the next month
    previousClose = close;
}

```

```
    return candlesticks;
}
```

In this function, It will take the Year and Month to display. This is done by the substring which is to look up the string character by character. Then the other process are similar to computeCandlestickData function.

After the calculation, the result is return to the filterByMonth function and it is displayed

```
// Daily Data
void WeatherApp::filterByDay(const std::vector<WeatherRow>& data) {
    std::string input;
    int year, month;

    while (true) {
        std::cout << "Enter year (1980-2019) or type 'exit' to quit: ";
        std::cin >> input;

        if (input == "exit") {
            std::cout << "Program exit, thank you for using!" << std::endl;
            exit(0);
        }

        try {
            year = std::stoi(input);
            if (year >= 1980 && year <= 2019) {
                break;
            } else {
                std::cerr << "Invalid year. Please enter a year between 1980
and 2019." << std::endl;
            }
        } catch (const std::invalid_argument&) {
            std::cerr << "Invalid input. Please enter a valid year or type
'exit' to quit." << std::endl;
        }
    }

    while (true) {
        std::cout << "Enter month (1-12) or type 'exit' to quit: ";
        std::cin >> input;

        if (input == "exit") {
            std::cout << "Program exit, thank you for using!" << std::endl;
            exit(0);
        }
    }
}
```

```

        try {
            month = std::stoi(input);
            if (month >= 1 && month <= 12) {
                break;
            } else {
                std::cerr << "Invalid month. Please enter a number between 1
and 12." << std::endl;
            }
        } catch (const std::invalid_argument&) {
            std::cerr << "Invalid input. Please enter a valid month or type
'exit' to quit." << std::endl;
        }
    }

    auto dailyCandlesticks =
WeatherAnalyzer::computeDailyCandlestickData(data, selectedCountry, year,
month);

    if (dailyCandlesticks.empty()) {
        std::cerr << "No data found for the given date and country." <<
std::endl;
        return;
    }

    CSVReader::adjustFirstOpenValue(dailyCandlesticks, 3);
    displayCandlesticks(dailyCandlesticks, 3);
}

```

```

// Computes daily candlestick data
std::vector<Candlestick> WeatherAnalyzer::computeDailyCandlestickData(
    const std::vector<WeatherRow>& data, const std::string& country, int year,
int month) {
    std::vector<Candlestick> dailyData;
    std::map<int, std::vector<double>> dailyTemperatures;

    // Some months dont' have 31 days
    auto daysInMonth = [](int year, int month) {
        // Calculate leap year
        if (month == 2) {
            return (year % 4 == 0 && year % 100 != 0) || (year % 400 == 0) ?
29 : 28;
        }
        return (month == 4 || month == 6 || month == 9 || month == 11) ? 30 :
31;
    };

    int maxDays = daysInMonth(year, month);
    for (const auto& row : data) {

```

```

        try {
            int rowYear = std::stoi(row.timestamp.substr(0, 4));
            int rowMonth = std::stoi(row.timestamp.substr(5, 2));
            int rowDay = std::stoi(row.timestamp.substr(8, 2));

            if (rowYear == year && rowMonth == month &&
row.values.find(country) != row.values.end()) {
                dailyTemperatures[rowDay].push_back(row.values.at(country));
            }
        } catch (const std::exception& e) {
            std::cerr << "Error processing with timestamp: " << row.timestamp
<< std::endl;
        }
    }

    double previousClose = 0.0;
    for (int day = 1; day <= maxDays; ++day) {
        double open = previousClose, high = 0.0, low = 0.0, close = 0.0;

        if (dailyTemperatures.count(day)) {
            computeStatistics(dailyTemperatures[day], high, low, close);
        }

        dailyData.emplace_back(
            std::to_string(year) + "-" + (month < 10 ? "0" : "") +
std::to_string(month) + "-" +
            (day < 10 ? "0" : "") + std::to_string(day),
            open, high, low, close);
        previousClose = close;
    }

    return dailyData;
}

```

For filter by day, it is a bit tricky compared to the other two because some months don't have 31 days. I have applied that logic as conditional statement as in computeDailyCandlestickData function. Then the other functions inside are similar to the previous two functions so I will skip that.

Except that for daily, user need to indicate, the year and the month of the daily data he wants to view.

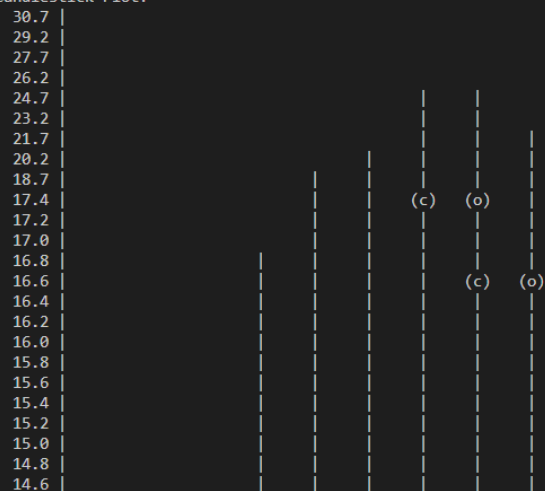
This is all the extra modifications for task 3 apart from task 1 and 2 implementation

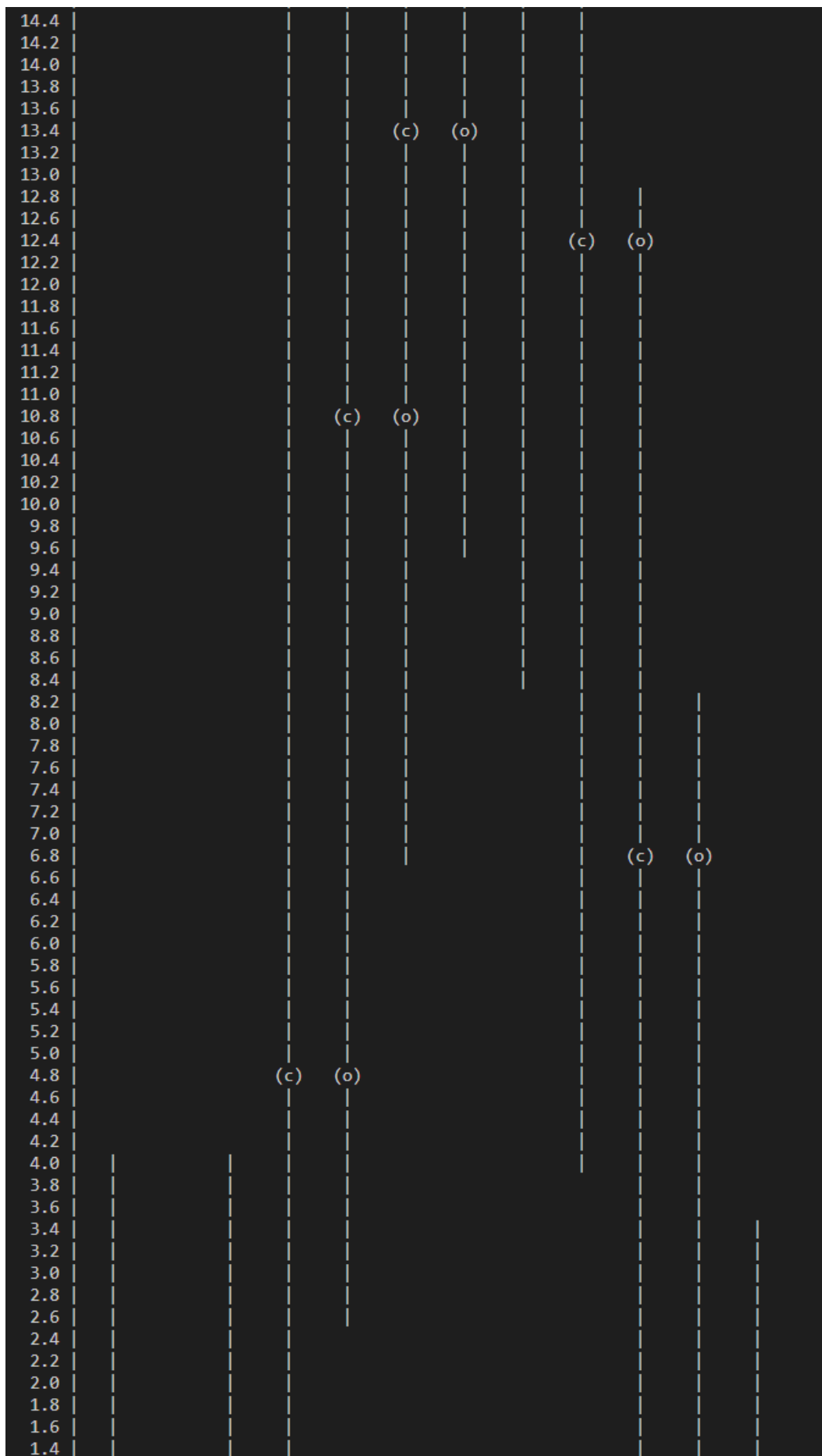
```
Back to country selection
Available Country Code for weather data: AT BE BG CH CZ DE DK EE ES FI FR GB GR HR HU IE IT LT LU LV NL NO PL PT RO SE SI SK
Enter the country code (or type 'exit' to quit, 'b' to go back): b
Select mode:
1. Display Data
2. Predict Yearly Data
Enter choice (1 or 2, or 'exit' to quit): afadfa
Invalid choice. Please enter 1, 2, or 'exit'.
Select mode:
1. Display Data
2. Predict Yearly Data
Enter choice (1 or 2, or 'exit' to quit): 1
Available Country Code for weather data: AT BE BG CH CZ DE DK EE ES FI FR GB GR HR HU IE IT LT LU LV NL NO PL PT RO SE SI SK
Enter the country code (or type 'exit' to quit, 'b' to go back): EE
```

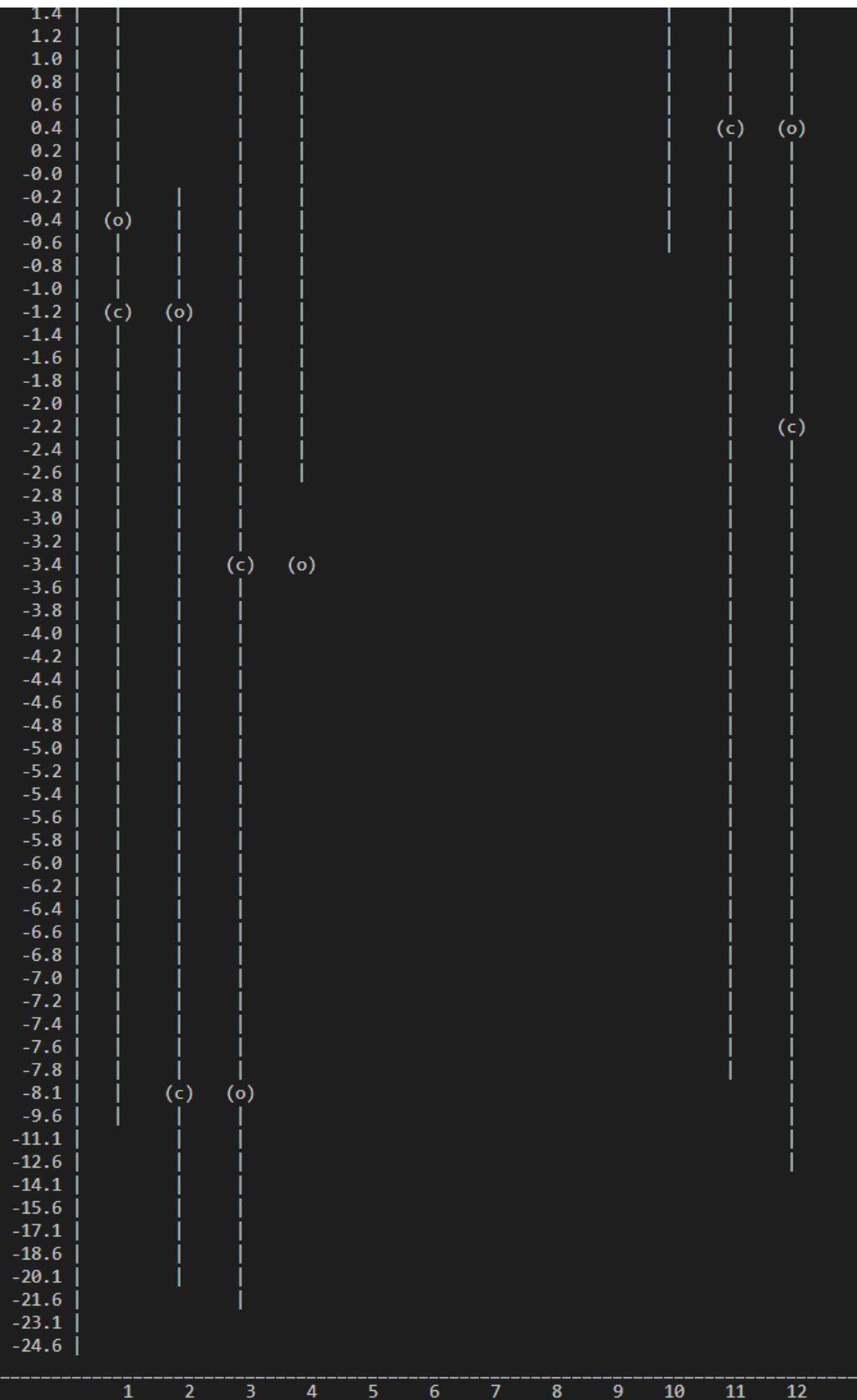
```
Select date range type:
1. Yearly
2. Monthly
3. Daily
4. Back to Country Selection
Enter choice (from 1 to 4 or type 'exit' to quit): 2
Enter year (1980-2019) or type 'exit' to quit: 1983
```

Date	Open	High	Low	Close
1983-01	-0.40	4	-10	-1.16
1983-02	-1.16	0	-20	-7.97
1983-03	-7.97	4	-21	-3.25
1983-04	-3.25	17	-3	4.87
1983-05	4.87	19	3	10.94
1983-06	10.94	20	7	13.41
1983-07	13.41	25	10	17.35
1983-08	17.35	26	8	16.64
1983-09	16.64	23	4	12.47
1983-10	12.47	13	-1	6.89
1983-11	6.89	8	-8	0.46
1983-12	0.46	4	-11	-2.23

Candlestick Plot:





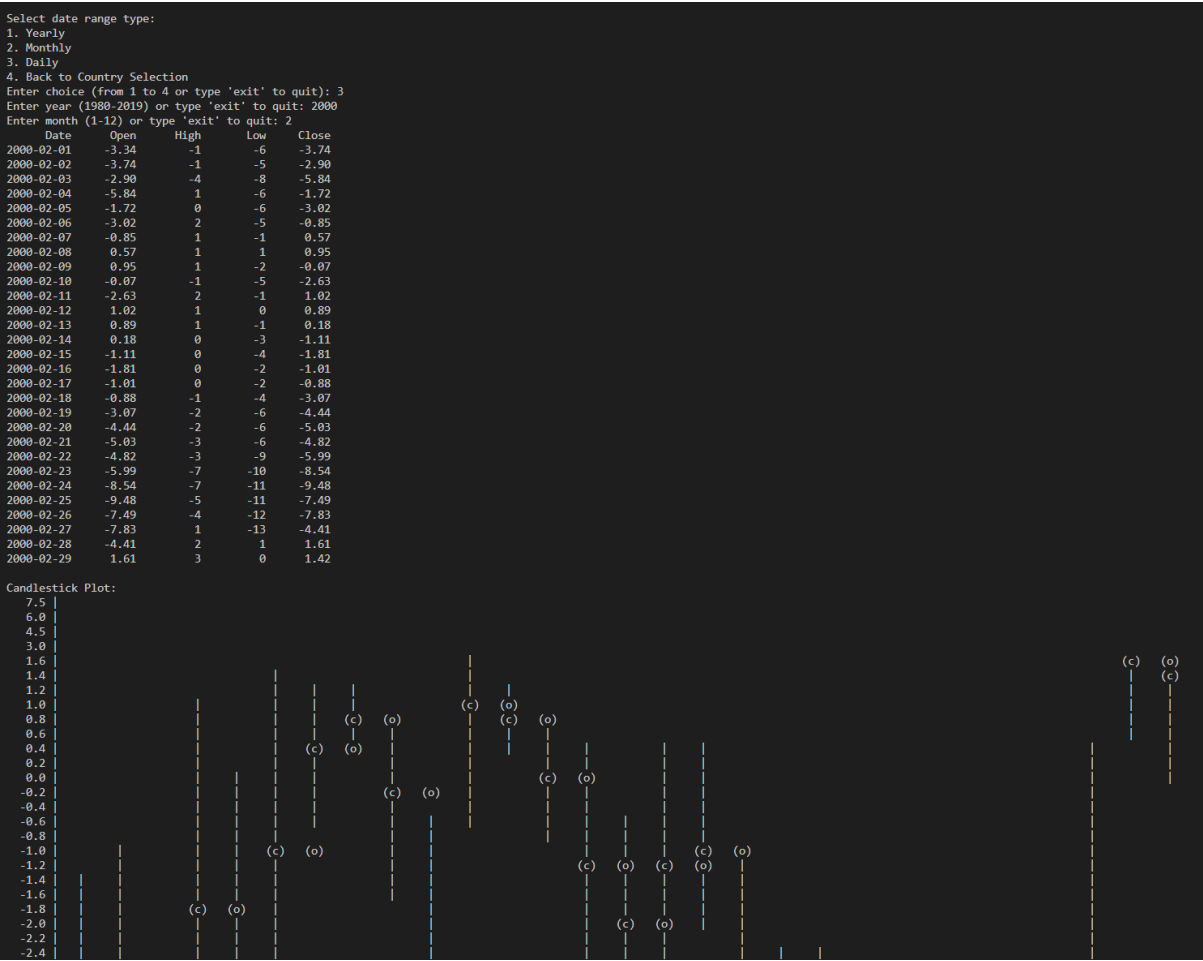


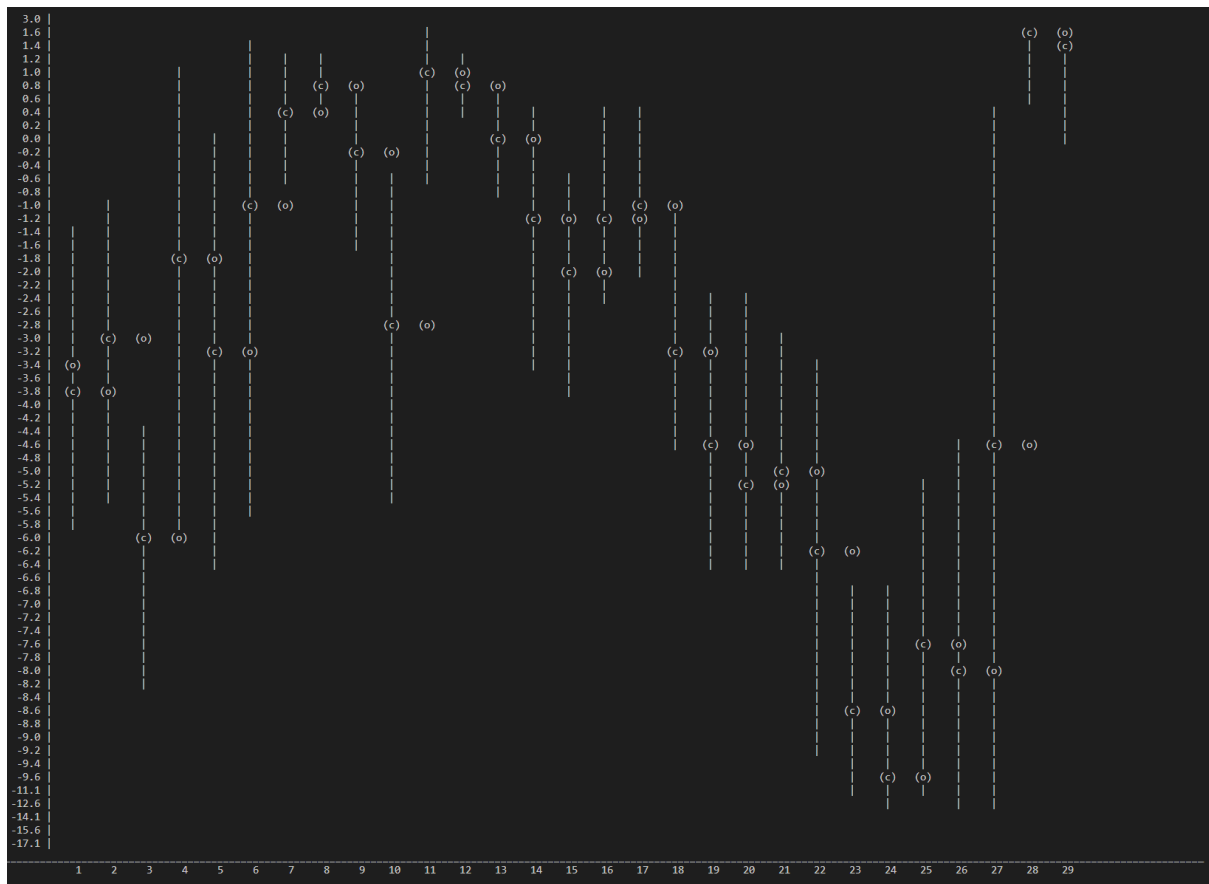
Select date range type:

1. Yearly
2. Monthly
3. Daily
4. Back to Country Selection

Enter choice (from 1 to 4 or type 'exit' to quit):

Those are an example output of monthly data and as you can see, some of the mean data have only a few gaps while some have huge gaps. So the graph is quite long here.





These are the output data for day sticks.

Task 4

For task 4, I only implement 1 prediction model, which is predicting the next year overall weather and the mean value based on past 8 years data. This model might also work for daily data but for monthly this might not work and here is why. I had applied weightage to the value of previous year and get the average to predict estimates mean value. For month, there are season changes from month to month so if we are using that model, it might be inaccurate because of season changes. (example: based on previous data, next month value is 5 but next month can be winter season and the value will be different)

```
double WeatherAnalyzer::predictYearlyWeather(const std::vector<Candlestick>&
candlesticks) {
    if (candlesticks.empty()) {
        throw std::runtime_error("No data available for prediction.");
        // Alternatively, you could return a default value, e.g.:
        // return 0.0;
    }
}
```

```

        std::vector<double> weights = {2, 5, 8, 11, 14, 17, 20, 23}; // Adjust
weight percentages
        size_t weightCount = weights.size();
        double weightedSum = 0.0, weightTotal = 0.0;

        for (size_t i = 0; i < candlesticks.size() && i < weightCount; ++i) {
            double weight = weights[weightCount - i - 1]; // Most recent year gets
highest weight
            weightedSum += candlesticks[candlesticks.size() - i - 1].close *
weight;
            weightTotal += weight;
        }

        return weightedSum / weightTotal; // Return predicted close value
    }

```

This is the code for that prediction and it simply add weightage to the mean values to calculate the estimation. Here, I prioritize the recent year because recent value will be more related to next weather data than old one.

```

void WeatherApp::handlePrediction(const std::vector<WeatherRow>& data) {
    while (true) {
        if (!selectCountry(data)) {
            return;
        }

        auto candlesticks = WeatherAnalyzer::computeCandlestickData(data,
selectedCountry);

        if (candlesticks.empty()) {
            std::cerr << "No data found for the selected country." <<
std::endl;
        } else {
            double prediction =
WeatherAnalyzer::predictYearlyWeather(candlesticks);
            std::cout << "\nPrediction for upcoming year:" << std::endl;
            // Above 0 is hot other value is cold
            std::cout << "Overall Weather: " << (prediction > 0 ? "Hot" :
"Cold") << std::endl;
            std::cout << "Predicted Close Value: " << std::fixed <<
std::setprecision(2) << prediction << std::endl;
        }

        std::string input;
        std::cout << "Type anything to continue or else enter ('b' to go back
to main menu or 'exit' to quit): ";
        std::cin >> input;
    }
}

```

```

        if (input == "b") {
            return;
        }

        if (input == "exit") {
            std::cout << "Program exit, thank you for using!" << std::endl;
            exit(0);
        }
    }
}

```

So, in menu, if the user choose second option, handlePrediction will be run and it will call predictYearlyWeather function.

This is all for my Task 4 and here is the prediction output

```

Back to country selection
Available Country Code for weather data: AT BE BG CH CZ DE DK EE ES FI FR GB GR HR HU IE IT LT LU LV NL NO PL PT RO SE SI SK
Enter the country code (or type 'exit' to quit, 'b' to go back): b
Select mode:
1. Display Data
2. Predict Yearly Data
Enter choice (1 or 2, or 'exit' to quit): 2
Available Country Code for weather data: AT BE BG CH CZ DE DK EE ES FI FR GB GR HR HU IE IT LT LU LV NL NO PL PT RO SE SI SK
Enter the country code (or type 'exit' to quit, 'b' to go back): EE

Prediction for upcoming year:
Overall Weather: Hot
Predicted Close Value: 6.55
Type anything to continue or else enter ('b' to go back to main menu or 'exit' to quit): 

```

```

Prediction for upcoming year:
Overall Weather: Hot
Predicted Close Value: 6.55
Type anything to continue or else enter ('b' to go back to main menu or 'exit' to quit): exit
Program exit, thank you for using!
PS C:\Users\phone\OneDrive - SIM - Singapore Institute of Management\Documents\SIM\Lessons\UOL Second Year\OOP (C++)\assignment\assignment 2.31> 

```